Lab Report: PS-Based LED-Switch Controller on Zynq-7000 SoC

Name: Mayur M Vernekar
Roll No: 152502014

Abstract

This experiment demonstrates the design and implementation of a hardware-software co-design system on the Zynq-7000 SoC (ZedBoard). The objective is to control the board's user LEDs using the DIP switches through the Processing System (PS). A custom AXI4-Lite peripheral was created to interface the Programmable Logic (PL) resources with the ARM Cortex-A9 processor.

1 Introduction

The Zynq-7000 architecture combines a dual-core ARM Cortex-A9 processor (Processing System or PS) with FPGA logic (Programmable Logic or PL). In this experiment, we utilize the Advanced eXtensible Interface (AXI) to create a communication channel between the PS (Master) and a custom peripheral (Slave). The application running on the PS performs memory-mapped read and write operations to control the hardware peripherals.

2 Methodology

2.1 Vivado Hardware Design

The hardware design was implemented using Xilinx Vivado according to the following procedure:

1. **Project Initialization:** A new RTL project was created in Vivado targeting the ZedBoard. The `ZYNQ7 Processing System` IP was added to the block design.

2. **Custom IP Generation:** A new AXI4 peripheral named `inputoutput` was created using the "Create and Package New IP" wizard. The interface was configured as AXI-Lite Slave.

3. **Verilog RTL Modifications:** The generated IP files were modified to include the necessary logic and ports.

   (a) **Editing the AXI Logic (Input_v1_0_S00_AXI):** Inside the main AXI logic file, the following changes were made:

   - Added `switches_in` and `leds_out` ports (Lines 17-19).

- Modified the slave register write logic (Lines 374-378) to include the test value:

```
// Inside slave register write logic
slv_reg2 <= 32'h0xaabbccdd; // Test value for software verification
```

- Modified the slave register read logic to capture switch inputs:

```
slv_reg0 <= {24'b0, switches_in};
```

- Added the assignment for LEDs at Line 402:

```
assign leds_out = slv_reg0[7:0];
```

(b) **Editing the IP Wrapper (InputOutput_v1_0):** The top-level wrapper file was edited to expose the ports to the block design:

- Added the ports in lines 18 and 19:

```
input wire [7:0] switches_in,
output wire [7:0] leds_out,
```

- Instantiated the ports in the module instantiation (Lines 73 and 74):

```
.switches_in(switches_in),
.leds_out(leds_out)
```

4. **Block Design Integration:** The modified IP was instantiated in the block design. The external ports `switches_in` and `leds_out` were created and connected to the custom IP. Block automation was run to connect the AXI interconnect.

5. **Constraints:** The `.xdc` file was updated to map the ports to the ZedBoard physical pins using the `LVCMOS33` I/O standard.

   - **LEDs:** Pins T22, T21, U22, U21, V22, W22, U19, U14.
   - **Switches:** Pins F22, G22, H22, F21, H19, H18, H17, M15.

2.2 Vitis Software Development

Following bitstream generation and hardware export, the software application was developed in Vitis:

1. **Application Code:** A C program was written to interface with the peripheral at base address `0x43c00000`. The code continuously reads the switch values and writes them back to the register:

```
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"

volatile unsigned int *addr = (volatile unsigned int *)0x43c00000;

```

```c
int main()
{
    init_platform();
    // Test write to register
    if(addr[2] == 0xaabbccdd) print("success");

    print("Hello World\n\r");
    print("Successfully ran Hello World application");

    // Infinite loop: Read switches (addr[1]) write to LEDs (addr[0])
    while(1) {
        addr[0] = addr[1];
    }

    cleanup_platform();
    return 0;
}
```

Listing 1: C Application Code

2. **Build Fix:** To resolve compilation errors on the Windows environment regarding wildcard expansions, the Makefile was modified:

   - Changed `LIBSOURCES= *.c` to `LIBSOURCES=$(wildcard *.c)`.

3 Implementation Results

3.1 Block Design

The final block design (Figure 1) demonstrates the integration of the Zynq Processing System with the custom `inputoutput` peripheral via the AXI interconnect.

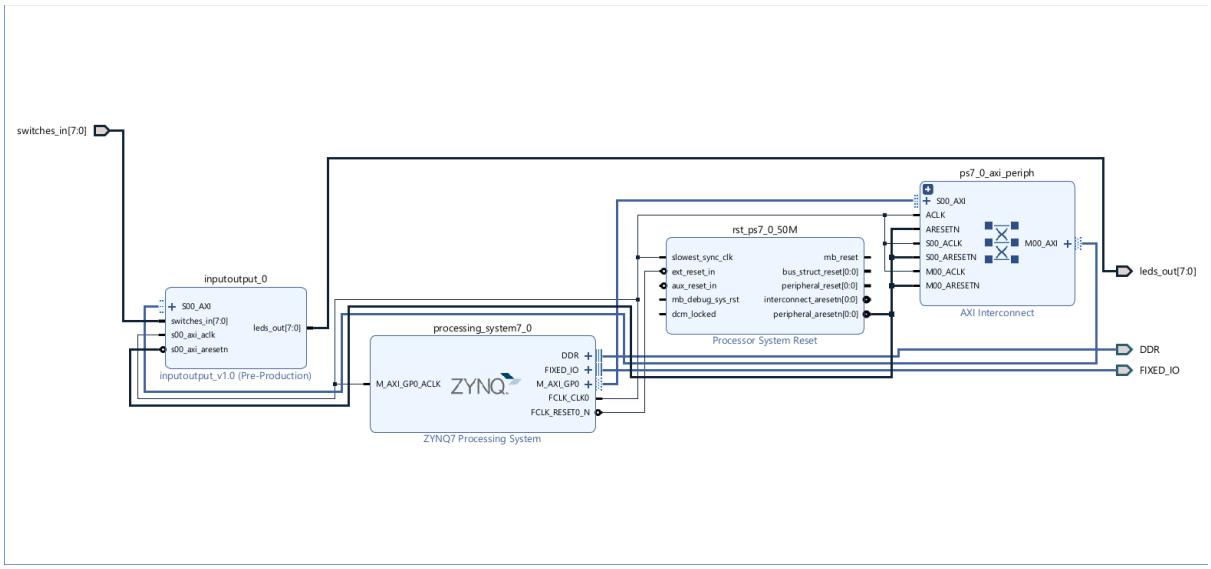

Figure 1: Vivado Block Design showing Zynq PS and AXI Peripheral.

3.2 Resource Utilization

The design resource usage was analyzed post-implementation. The custom peripheral utilizes minimal FPGA fabric resources, consuming less than 1% of the available LUTs and Flip-Flops.

Resource	Utilization	Available	Utilization %
LUT (Look-Up Tables)	515	53,200	0.97%
FF (Flip-Flops)	708	106,400	0.67%
IO (Input/Output)	16	200	8.00%

Table 1: FPGA Resource Utilization.

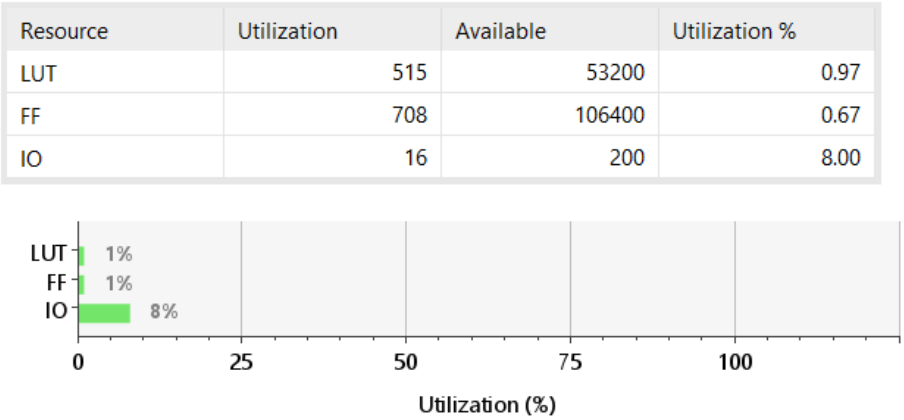

Figure 2: Graphical Representation of Resource Utilization.

3.3 Device Implementation

Figure 3 illustrates the physical placement of the logic on the Zynq-7000 fabric. The logic is placed near the I/O banks corresponding to the constraints.

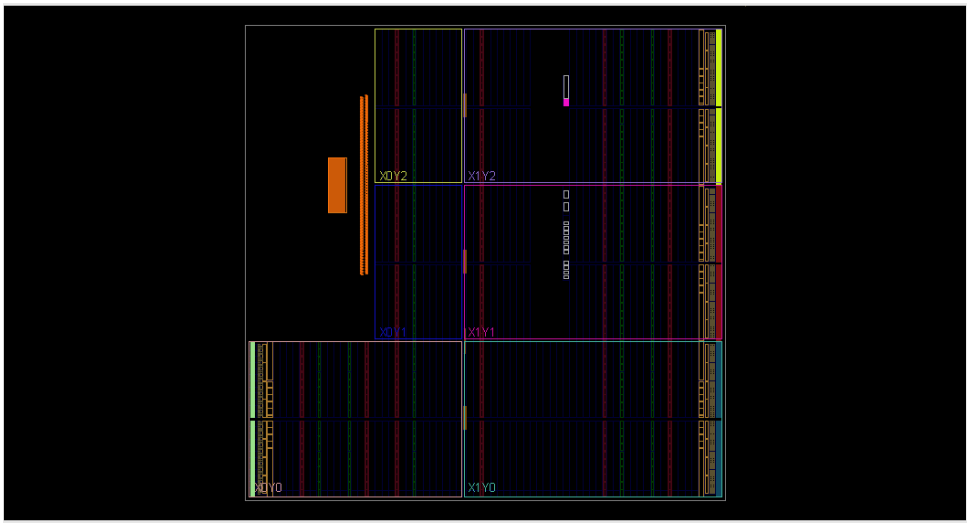

Figure 3: Implemented Device View.

3.4 Hardware Verification

The design was verified on the physical ZedBoard hardware. The software application successfully read the state of the DIP switches and updated the LEDs in real-time. Figure 4 shows the board during operation, where active switches correspond to illuminated LEDs.

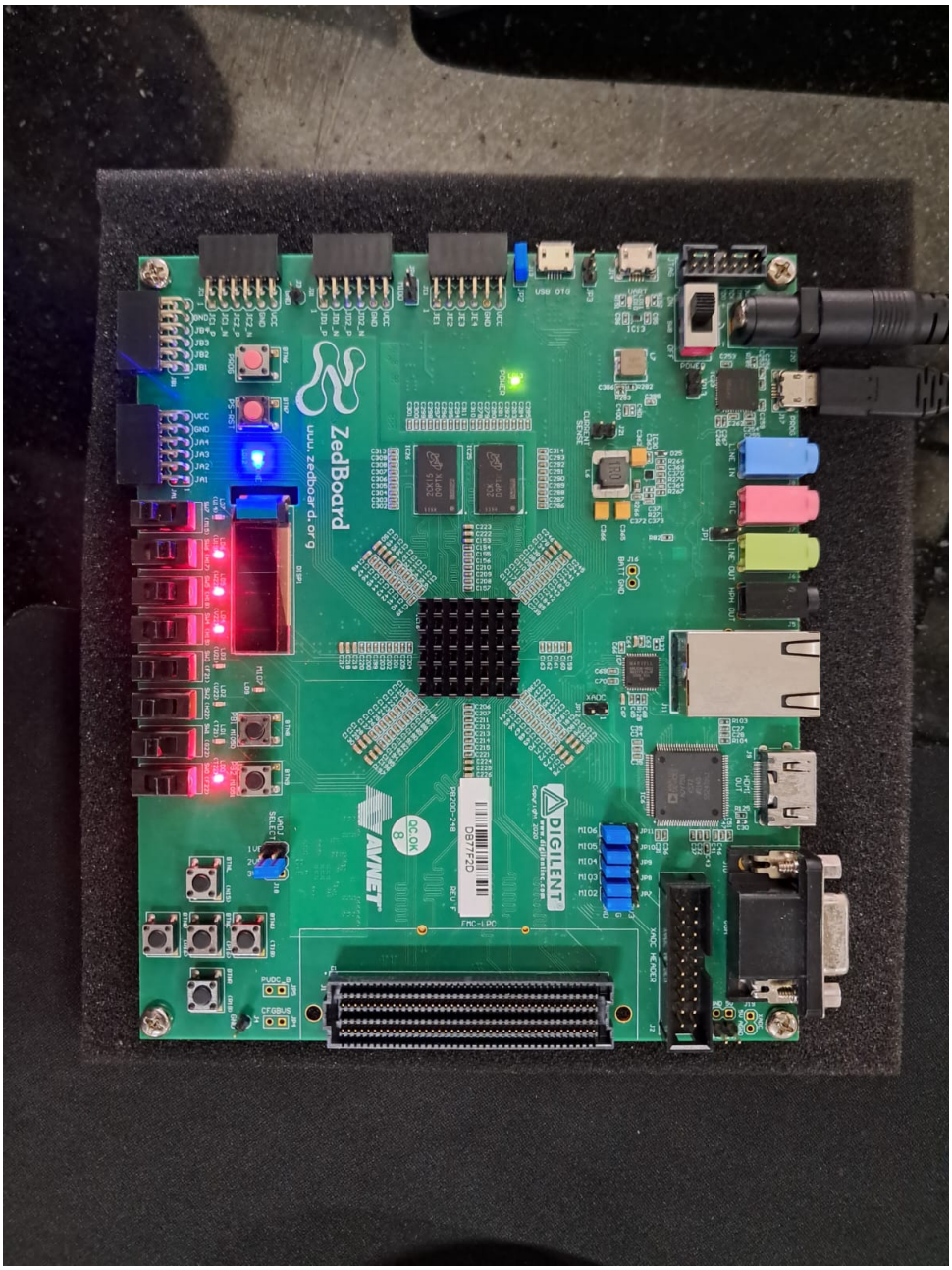

Figure 4: Hardware verification on ZedBoard showing active LEDs controlled by switches.

4 Conclusion

The experiment successfully established a hardware-software interface on the Zynq-7000 SoC. By mapping custom hardware logic to the processor's memory space via the AXI4-Lite interface, we demonstrated how software can directly control physical hardware

peripherals. The correct operation was verified on the ZedBoard, where toggling the DIP switches resulted in corresponding changes in the user LEDs.